

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

PROJECT REPORT

Network Intrusion Detection System Using Machine Learning

Nguyen Duy Phuc
phuc.nd235616@sis.hust.edu.vn

Program: Cyber Security

Supervisor: Dr. Tran Hai Anh

Department: Computer Engineering

School: School of Information and Communications Technology

HANOI, 2026

Abstract

This project develops a Machine Learning-based Network Intrusion Detection System (NIDS) for classifying network traffic flows into normal and malicious categories. The system is built around the CICIDS2017 dataset, a widely used benchmark dataset for intrusion detection research. The project implements a full training and evaluation pipeline, including dataset loading, label grouping, data cleaning, feature scaling, class imbalance handling, model training, model comparison, model persistence, and offline PCAP-based inference.

The main task is multi-class classification. The final label space includes **Normal**, **DoS**, **DDoS**, **PortScan**, **BruteForce**, **Bot**, and **Infiltration**. Four machine learning models are evaluated: Logistic Regression, K-Nearest Neighbors, Random Forest, and XGBoost. Because CICIDS2017 is highly imbalanced, especially for minority classes such as **Bot** and **Infiltration**, the project also evaluates model behavior before and after conservative imbalance handling. The imbalance strategy combines Random Under-Sampling for the majority **Normal** class with light SMOTE for selected minority classes.

Experimental results show that Random Forest without imbalance handling achieves the best full-dataset Test Macro-F1 score of 0.9378, while XGBoost with conservative imbalance handling shows the clearest improvement on minority-class detection, increasing Test Macro-F1 from 0.8243 to 0.9215 and improving **Infiltration** F1 from 0.00 to 0.71. Logistic Regression performs poorly on rare attack classes, while KNN achieves moderate full-dataset performance but is computationally expensive during inference.

The project also supports offline PCAP testing. PCAP files are converted into CICIDS-like flow features using a patched CICFlowMeter-based extractor or a Scapy fallback. The trained model then predicts traffic classes from the extracted flows. This inference workflow demonstrates how the trained NIDS can be applied beyond static CSV evaluation, while also showing the practical limitations caused by domain shift, feature extraction differences, and the mismatch between benchmark flow data and external packet captures.

Contents

Abstract	i
1 Introduction	1
1.1 Background	1
1.2 Project Motivation	1
1.3 Project Scope	1
2 Project Structure	2
2.1 Data Directory	2
2.2 Model Directories	2
2.3 Source Code	3
3 Dataset	3
3.1 Dataset Source	3
3.2 CSV Files Used	3
3.3 Feature Type	4
3.4 Original Class Distribution	4
3.5 Dataset Imbalance	4
4 Label Grouping	4
5 Preprocessing Pipeline	5
5.1 Column Name Normalization	5
5.2 Metadata Removal	5
5.3 Missing and Infinite Values	6
5.4 Numeric Conversion	6
5.5 Train/Validation/Test Split	6
5.6 Label Encoding and Feature Scaling	6
6 Class Imbalance Handling	7
6.1 Strategy	7

6.2	Normal Under-Sampling	7
6.3	Conservative SMOTE	7
6.4	Training Distribution After Imbalance Handling	7
7	Machine Learning Models	8
7.1	Logistic Regression	8
7.2	K-Nearest Neighbors	8
7.3	Random Forest	8
7.4	XGBoost	9
8	Training Pipeline	9
8.1	Training Commands	10
9	Reproducibility and Artifacts	10
9.1	Main Dependency Versions	10
9.2	Saved Diagnostic Files	11
10	Evaluation Metrics	11
10.1	Macro-F1	11
10.2	Weighted-F1	11
11	Full Dataset Results	12
11.1	Overall Results	12
11.2	Impact of Imbalance Handling	12
11.3	Per-Class Test F1	13
11.4	XGBoost Confusion Matrix and Diagnostics	13
11.5	Additional Confusion Matrices for Other Models	14
11.6	Interpretation	15
12	Model Selection	15
13	PCAP Inference	16
13.1	Inference Flow	16

13.2 Default Model Directory	16
13.3 PCAP Commands	16
13.4 PCAP Test Results	17
13.4.1 task3.dos_victim.pcap	17
13.4.2 Other PCAP Files	17
13.5 PCAP Interpretation	17
14 Usage Guide	18
14.1 Install Dependencies	18
14.2 Train a Model	18
14.3 Test a PCAP File	18
14.4 Test With a Specific Model	18
15 Limitations	18
16 Future Work	19
17 Conclusion	19

1 Introduction

1.1 Background

Network Intrusion Detection Systems monitor network traffic and detect suspicious or malicious activity. Traditional intrusion detection systems often rely on signature-based rules. Although signature-based methods can detect known attacks reliably, they usually struggle with unseen attack patterns, modified payloads, and zero-day behavior.

Machine learning offers a different approach. Instead of checking only fixed rules, a model learns statistical patterns from historical network traffic. Once trained, it can classify new traffic based on flow-level behavior such as packet counts, byte counts, packet lengths, inter-arrival times, protocol flags, and traffic direction statistics.

This project focuses on flow-based intrusion detection. A flow represents a communication session or a group of packets sharing common properties such as source, destination, ports, and protocol. Flow-based features are more compact than raw packets and are commonly used in intrusion detection datasets such as CICIDS2017.

1.2 Project Motivation

The goal of this project is not only to train a high-accuracy model, but also to understand model behavior under dataset imbalance and evaluate whether trained models can be used on offline PCAP traffic.

The project investigates the following questions:

- Can machine learning classify common network attacks from CICIDS2017 flow features?
- How does class imbalance affect the result?
- Does imbalance handling improve all models equally?
- Which model is most suitable for benchmark reporting?
- Which model is most useful for PCAP inference?
- Can a model trained on CICIDS2017 generalize to external PCAP files?

1.3 Project Scope

The project includes CICIDS2017 CSV dataset loading, multi-class label grouping, data cleaning, preprocessing, train/validation/test splitting, conservative dataset imbalance

handling, model training, model comparison, artifact saving, and offline PCAP inference through flow extraction.

The project no longer includes rule-based log analysis. The previous log analysis workflow was removed to keep the project focused on machine learning and PCAP-based NIDS inference.

2 Project Structure

The project is organized into a small number of main directories:

```
NIDS---Network-Intrusion-Detection-System/
|-- data/
| |-- raw/CICIDS2017/
| '-- pcap/
|-- models/final/
|-- experiments/models/
|-- outputs/
|-- reports/training_runs/
|-- src/
| |-- data/
| |-- models/
| |-- pipeline/
| '-- utils/
|-- live_analyze.py
|-- main.py
|-- requirements.txt
|-- README.md
'-- REPORT.md
```

2.1 Data Directory

The `data/raw/CICIDS2017/` folder contains the CICIDS2017 CSV flow files used for training and evaluation. The `data/pcap/` folder contains offline PCAP files used for inference testing.

2.2 Model Directories

The `models/final/` directory is the default inference directory. It is generated locally after training and is intentionally not committed to Git because model artifacts can be

large and environment-specific. It contains the model artifacts used by PCAP inference:

```
model.pkl  
scaler.pkl  
label_encoder.pkl  
features.pkl
```

The `experiments/models/` folder stores artifacts from different experiment runs. It is separated from `models/final/` to keep the main model directory clean. These experiment artifacts are also local runtime outputs and are not required in the GitHub repository.

2.3 Source Code

The `src/` directory contains reusable source code. The `src/data/` package handles loading, cleaning, label mapping, and imbalance handling. The `src/models/` package contains model definitions and prediction helpers. The `src/pipeline/` package contains the end-to-end training pipeline.

3 Dataset

3.1 Dataset Source

The project uses the CICIDS2017 flow dataset. The dataset is stored in:

```
data/raw/CICIDS2017/
```

The loaded dataset contains 2,830,743 rows and 79 columns including the label column.

3.2 CSV Files Used

The full dataset is loaded from eight CICIDS2017 CSV files:

- `Monday-WorkingHours.pcap_ISCX.csv`
- `Tuesday-WorkingHours.pcap_ISCX.csv`
- `Wednesday-workingHours.pcap_ISCX.csv`
- `Thursday-WorkingHours-Morning-WebAttacks.pcap_ISCX.csv`
- `Thursday-WorkingHours-Afternoon-Infiltration.pcap_ISCX.csv`

- `Friday-WorkingHours-Morning.pcap_ISCX.csv`
- `Friday-WorkingHours-Afternoon-PortScan.pcap_ISCX.csv`
- `Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv`

3.3 Feature Type

The dataset uses flow-level features rather than raw packets. Example feature groups include flow duration, forward and backward packet counts, byte statistics, packet length statistics, flow rate, inter-arrival time statistics, TCP flag counts, header length statistics, and active/idle time statistics.

3.4 Original Class Distribution

Table 1: Original class distribution after label grouping.

Class	Samples
Normal	2,273,097
DoS	252,661
PortScan	158,930
DDoS	128,027
BruteForce	13,835
Bot	1,966
Infiltration	47

3.5 Dataset Imbalance

The dataset is highly imbalanced. **Normal** traffic dominates the dataset, while **Infiltration** has only 47 samples. Accuracy and Weighted-F1 can therefore be misleading. A model can perform very well on majority classes while failing rare attacks. For this reason, Macro-F1 is emphasized throughout the evaluation.

4 Label Grouping

The raw CICIDS2017 labels are mapped into broader attack categories.

Table 2: Label grouping used in preprocessing.

Original Label	Grouped Label
BENIGN, Benign, Normal	Normal
DoS Hulk, DoS GoldenEye, DoS Slowloris, DoS Slowhttptest, DoS	DoS
DDoS	DDoS
PortScan	PortScan
FTP-Patator, SSH-Patator, BruteForce	BruteForce
Bot	Bot
Web Attack Brute Force, Web Attack XSS, Web Attack Sql Injection	Web
Infiltration, Heartbleed	Infiltration

In the final full-dataset experiments, the observed classes are Normal, DoS, DDoS, PortScan, BruteForce, Bot, and Infiltration.

5 Preprocessing Pipeline

5.1 Column Name Normalization

Column names are stripped to remove leading or trailing spaces:

```
df_clean.columns = df_clean.columns.str.strip()
```

5.2 Metadata Removal

Metadata columns are removed before training:

```
Flow ID
Source IP
Source Port
Destination IP
Destination Port
Protocol
Timestamp
```

Equivalent fields from PCAP flow extractors are also removed, such as `flow_id`, `src_ip`, `src_port`, `dst_ip`, `dst_port`, `protocol`, and `timestamp`. These fields identify traffic

endpoints rather than general behavior and could cause the model to learn dataset-specific identifiers.

5.3 Missing and Infinite Values

Invalid numerical values are handled as follows:

```
df_clean.replace([np.inf, -np.inf], np.nan, inplace=True)
df_clean.fillna(0, inplace=True)
```

5.4 Numeric Conversion

All feature columns are converted to numeric values. Failed conversions become `NaN` and are later filled with zero.

5.5 Train/Validation/Test Split

The cleaned dataset is split into train, validation, and test subsets.

Table 3: Dataset split ratio.

Split	Ratio
Train	70%
Validation	10%
Test	20%

The split is stratified when possible so that class proportions are preserved.

5.6 Label Encoding and Feature Scaling

Text labels are encoded using `LabelEncoder`. Features are scaled using `StandardScaler`. The scaler is fitted only on the training set, while validation, test, and inference data only use `transform`. This avoids data leakage.

6 Class Imbalance Handling

6.1 Strategy

The imbalance handling function uses a conservative strategy:

1. Random under-sampling for the majority `Normal` class.
2. Light SMOTE for selected minority classes.
3. No aggressive oversampling for extremely rare classes.

6.2 Normal Under-Sampling

The `Normal` class is reduced to at most 300,000 training samples:

```
target_normal = min(300000, normal_count)
```

This reduces majority-class dominance while still keeping a large number of real normal samples.

6.3 Conservative SMOTE

Table 4: Conservative SMOTE strategy.

Condition	Strategy
Class has fewer than 100 samples	Do not increase
Bot	Increase up to 1.5x, capped at 2,500
Class has fewer than 2,000 samples	Increase up to 1.5x, capped at 4,000
Class has fewer than 10,000 samples	Increase up to 1.25x, capped at 12,000
Large classes	Keep unchanged

6.4 Training Distribution After Imbalance Handling

Table 5: Training distribution after imbalance handling.

Class	Training Samples
Normal	300,000
DoS	176,863

PortScan	111,251
DDoS	89,618
BruteForce	12,000
Bot	2,064
Infiltration	33

The original `Infiltration` count is 47 in the full dataset. The value 33 appears after the train/validation/test split because imbalance handling is applied only to the training set.

7 Machine Learning Models

7.1 Logistic Regression

Logistic Regression is used as a linear baseline.

```
solver = saga
max_iter = 300
tol = 1e-3
n_jobs = -1
random_state = 42
```

It performs poorly on rare attack classes and produces convergence warnings on the full dataset.

7.2 K-Nearest Neighbors

KNN is used as a distance-based baseline.

```
n_neighbors = 5
n_jobs = -1
```

KNN achieves moderate Macro-F1 on the full dataset, but inference is slow because prediction requires distance comparisons against a large training set.

7.3 Random Forest

Random Forest is a tree-based ensemble model.

```
n_estimators = 100
n_jobs = -1
```

```
random_state = 42
```

It achieves the best full-dataset Test Macro-F1 on the internal CICIDS2017 split.

7.4 XGBoost

XGBoost is the main boosted tree model.

```
n_estimators = 600
max_depth = 4
learning_rate = 0.04
subsample = 0.8
colsample_bytree = 0.75
reg_alpha = 2.0
reg_lambda = 8.0
gamma = 2.0
min_child_weight = 10
max_delta_step = 1
objective = multi:softmax
eval_metric = merror
tree_method = hist
n_jobs = -1
random_state = 42
```

When XGBoost is trained with imbalance handling, clipped balanced sample weights are also used:

```
min weight = 0.75
max weight = 2.00
```

XGBoost shows the clearest improvement after imbalance handling and is the most sensitive model in the tested PCAP model-only workflow.

8 Training Pipeline

The full training process is:

```
Load CICIDS2017 CSV files
Clean data and map labels
Split into train/validation/test
Apply imbalance handling if enabled
Encode labels
```

```
Scale features
Train selected model
Evaluate validation set
Evaluate test set
Run sampled cross-validation if possible
Save model artifacts
```

8.1 Training Commands

Train XGBoost with imbalance handling:

```
python3 main.py --train data/raw/CICIDS2017 --model xgb
```

Train Random Forest:

```
python3 main.py --train data/raw/CICIDS2017 --model rf
```

Train without imbalance handling:

```
python3 main.py --train data/raw/CICIDS2017 --model rf --no-imbalance
```

Train and save artifacts to a custom directory:

```
python3 main.py --train data/raw/CICIDS2017 --model xgb --model-dir models/
    final
```

9 Reproducibility and Artifacts

The experiment results in this report are based on the committed source code and the saved training summaries under `reports/training_runs/`. Raw CICIDS2017 CSV files, trained model pickle files, PCAP files, and runtime outputs are kept as local artifacts because they can be large and environment-specific.

9.1 Main Dependency Versions

Table 6: Main Python dependency versions used by the project.

Package	Version
pandas	2.2.2
numpy	1.26.4

scikit-learn	1.5.1
joblib	1.4.2
xgboost	2.1.1
imbalanced-learn	0.14.1
scapy	2.5.0
cicflowmeter	0.1.6

9.2 Saved Diagnostic Files

The following small diagnostic files are kept in the GitHub repository because they support the report tables directly:

```
reports/final/xgb_confusion_matrix.csv
reports/final/rf_no_imbalance_confusion_matrix.csv
reports/final/lr_no_imbalance_confusion_matrix.csv
reports/final/xgb_feature_importance.csv
```

10 Evaluation Metrics

The project uses Precision, Recall, F1-score, Macro-F1, Weighted-F1, and sampled cross-validation where applicable.

10.1 Macro-F1

Macro-F1 averages F1-score across all classes equally. It is the most important metric for this project because it reveals whether minority classes are being detected.

10.2 Weighted-F1

Weighted-F1 averages F1-score by class support. Because **Normal** has very high support, Weighted-F1 can remain high even if rare classes perform poorly.

11 Full Dataset Results

11.1 Overall Results

Table 7: Full-dataset validation and test results.

Model	Imb.	Val Macro	Val W-F1	Test Macro	Test W-F1	CV W-F1
LR	No	0.4788	0.9224	0.4791	0.9224	Skipped
LR	Yes	0.5030	0.8940	0.4554	0.8938	0.8821 ± 0.0018
KNN	No	0.8818	0.9981	0.8769	0.9982	0.9834 ± 0.0011
KNN	Yes	0.8252	0.9916	0.8378	0.9914	Skipped
XGB	No	0.8682	0.9987	0.8243	0.9988	Skipped
XGB	Yes	0.9459	0.9984	0.9215	0.9985	0.9981 ± 0.0002
RF	No	0.9536	0.9988	0.9378	0.9987	Skipped
RF	Yes	0.9497	0.9983	0.9280	0.9985	0.9979 ± 0.0002

In Table 7, LR means Logistic Regression, XGB means XGBoost, and RF means Random Forest.

The word *Skipped* in the CV column means that only the sampled 5-fold cross-validation step was skipped. The model was still trained and evaluated on the validation and test sets. CV was skipped when the stratified CV sample contained too few examples of the rarest class for a reliable 5-fold split. For example, in several no-imbalance runs the rarest class had only 3 samples in the CV sample, and in the KNN imbalance run the rarest class had only 1 sample in the smaller CV sample used to reduce KNN computation cost.

11.2 Impact of Imbalance Handling

The table below summarizes how conservative imbalance handling changes the full-dataset Test Macro-F1 score. This comparison is important because the project is not optimized only for overall accuracy; it also studies whether rare attack detection improves.

Table 8: Effect of imbalance handling on Test Macro-F1.

Model	No Imb.	With Imb.	Change	Interpretation
Logistic Regression	0.4791	0.4554	-0.0237	Slightly worse; weak baseline
KNN	0.8769	0.8378	-0.0391	Worse due to density changes
XGBoost	0.8243	0.9215	+0.0972	Strong improvement
Random Forest	0.9378	0.9280	-0.0098	Slightly lower but still strong

This table shows that imbalance handling does not improve every model equally. It is most useful for XGBoost, where the rare **Infiltration** class improves from $F1 = 0.00$ to $F1 = 0.71$. Random Forest remains the strongest internal benchmark model, but XGBoost provides the clearest evidence that conservative imbalance handling can improve minority-class detection.

11.3 Per-Class Test F1

Table 9: Per-class Test F1 scores.

Model	Imb.	Bot	BruteForce	DDoS	DoS	Infiltration	Normal	PortScan
Logistic Regression	No	0.00	0.00	0.75	0.78	0.00	0.96	0.87
Logistic Regression	Yes	0.00	0.00	0.69	0.76	0.00	0.93	0.80
KNN	No	0.70	0.99	1.00	1.00	0.46	1.00	1.00
KNN	Yes	0.55	0.91	1.00	0.99	0.46	0.99	0.96
XGBoost	No	0.78	1.00	1.00	1.00	0.00	1.00	1.00
XGBoost	Yes	0.75	1.00	1.00	1.00	0.71	1.00	1.00
Random Forest	No	0.77	1.00	1.00	1.00	0.80	1.00	1.00
Random Forest	Yes	0.79	1.00	1.00	1.00	0.71	1.00	1.00

11.4 XGBoost Confusion Matrix and Diagnostics

The project also saves additional XGBoost diagnostics. The confusion matrix below corresponds to the XGBoost model with conservative imbalance handling. Rows represent true labels and columns represent predicted labels. This matrix uses the same 565,713-flow test set as the other confusion matrices.

Table 10: XGBoost confusion matrix with conservative imbalance handling.

True / Pred.	Bot	BruteForce	DDoS	DoS	Infiltration	Normal	PortScan
Bot	383	0	0	0	0	10	0
BruteForce	0	2758	0	1	0	8	0
DDoS	0	0	25600	0	0	6	0
DoS	0	0	0	50514	0	18	0
Infiltration	0	0	0	1	5	3	0
Normal	247	13	40	308	0	453804	208
PortScan	0	0	0	7	0	4	31775

The confusion matrix shows that most high-support attack classes are classified correctly. The rare **Infiltration** class remains difficult because the test set contains only 9 samples.

11.5 Additional Confusion Matrices for Other Models

The XGBoost matrix is the main diagnostic matrix because XGBoost is the model that benefits most from imbalance handling. However, Random Forest and Logistic Regression are also important for comparison. Random Forest represents the strongest internal CICIDS2017 benchmark, while Logistic Regression represents the weakest linear baseline.

The following two matrices are therefore included to show how the strongest benchmark model and the baseline model behave on the same test split. KNN is still compared through the overall result table and per-class F1 table, which already summarize its class-level behavior. The matrix CSV files are saved under `reports/final/` so the table values can be checked directly.

Table 11: Random Forest confusion matrix without imbalance handling.

True / Pred.	Bot	BruteForce	DDoS	DoS	Infiltration	Normal	PortScan
Bot	271	0	0	0	0	122	0
BruteForce	0	2754	0	0	0	13	0
DDoS	0	0	25591	0	0	15	0
DoS	0	0	1	50375	0	156	0
Infiltration	0	0	0	0	6	3	0
Normal	36	0	10	139	0	454237	198
PortScan	0	0	0	4	0	4	31778

Table 12: Logistic Regression confusion matrix without imbalance handling.

True / Pred.	Bot	BruteForce	DDoS	DoS	Infiltration	Normal	PortScan
Bot	0	0	0	0	0	281	112
BruteForce	0	0	0	0	0	1977	790
DDoS	0	0	15576	847	0	9183	0
DoS	0	0	263	34139	0	16130	0
Infiltration	0	0	1	1	0	7	0
Normal	0	2	129	2042	0	443846	8601
PortScan	0	0	8	12	0	205	31561

These matrices explain why Random Forest achieves the highest internal Macro-F1 while Logistic Regression remains a weak baseline. Random Forest correctly classifies most high-support attack classes and even detects most **Infiltration** samples. Logistic Regression collapses rare classes such as **Bot**, **BruteForce**, and **Infiltration** into majority classes, which is why its Macro-F1 remains low.

Table 13: Top XGBoost feature importance values.

Rank	Feature	Importance
1	Subflow Fwd Packets	0.0936
2	Bwd Packets/s	0.0791
3	act_data_pkt_fwd	0.0777
4	Total Fwd Packets	0.0688
5	Bwd Packet Length Std	0.0609
6	Min Packet Length	0.0583
7	Fwd Packet Length Mean	0.0435
8	Avg Fwd Segment Size	0.0434
9	Idle Max	0.0359
10	Subflow Bwd Bytes	0.0348

11.6 Interpretation

Random Forest without imbalance handling achieves the best internal CICIDS2017 Test Macro-F1 of 0.9378. XGBoost benefits the most from imbalance handling, improving Test Macro-F1 from 0.8243 to 0.9215 and improving **Infiltration** F1 from 0.00 to 0.71. KNN performs worse after imbalance handling because it is sensitive to sample density. Logistic Regression is the weakest model and should be treated only as a baseline.

12 Model Selection

For internal CICIDS2017 evaluation, the strongest model is Random Forest without imbalance handling. For imbalance-focused analysis and PCAP-oriented testing, the preferred model is XGBoost with conservative imbalance handling.

Table 14: Model selection decision matrix.

Goal	Recommended Model	Reason
Best internal CICIDS2017 benchmark score	Random Forest without imbalance handling	Highest full-dataset Test Macro-F1 score, 0.9378
Best imbalance-handling story	XGBoost with conservative imbalance handling	Test Macro-F1 improves from 0.8243 to 0.9215 and Infiltration F1 improves from 0.00 to 0.71

Most practical PCAP-oriented model in tested files	XGBoost with conservative imbalance handling	More sensitive to attack-like flow patterns in model-only PCAP inference
Simple baseline	Logistic Regression	Useful for comparison, but weak on rare classes
Distance-based baseline	KNN	Useful comparison model, but costly for large-scale inference

13 PCAP Inference

13.1 Inference Flow

```

Input PCAP
Extract flow features
Normalize feature columns
Clean data
Align columns with training feature list
Scale using saved scaler
Predict using saved model
Decode labels
Print NIDS report

```

13.2 Default Model Directory

By default, PCAP testing uses:

```
models/final/
```

Required files:

```

model.pkl
scaler.pkl
label_encoder.pkl
features.pkl

```

13.3 PCAP Commands

Default model:

```
python3 main.py --pcap data/pcap/test1.pcap
```

Specific model:

```
python3 main.py --pcap data/pcap/task3.dos_victim.pcap --model-dir experiments/  
models/xgb_with_imbalance
```

13.4 PCAP Test Results

13.4.1 task3.dos_victim.pcap

This file extracted 41,830 flows using `cicflowmeter-patched`.

Table 15: PCAP model comparison on task3.dos_victim.pcap.

Model	Normal	DoS	Interpretation
Logistic Regression	41,830	0	All flows predicted Normal
KNN	41,830	0	All flows predicted Normal
Random Forest	41,830	0	All flows predicted Normal
XGBoost with imbalance	36,368	5,462	Predicted DoS-like flows

13.4.2 Other PCAP Files

Table 16: Other PCAP test results.

PCAP	Extracted Flows	Result
task1.dos_victim.pcap	224	All models predicted Normal
task3.dos_attacker.pcap	43,525	All models predicted Normal
task1.dos_attacker.pcap	5,114	All models predicted Normal

13.5 PCAP Interpretation

The PCAP results show that high performance on the CICIDS2017 internal split does not guarantee strong generalization to external PCAP traffic. Domain shift and flow extraction differences matter. XGBoost with imbalance handling is the most useful model in the tested PCAP workflow because it is more sensitive to attack-like flow patterns than the other tested models. However, these results should be interpreted as model predictions, not as confirmed ground-truth labels.

14 Usage Guide

14.1 Install Dependencies

```
pip install -r requirements.txt
```

14.2 Train a Model

```
python3 main.py --train data/raw/CICIDS2017 --model xgb --model-dir models/  
final
```

14.3 Test a PCAP File

```
python3 main.py --pcap data/pcap/test1.pcap
```

14.4 Test With a Specific Model

```
python3 main.py --pcap data/pcap/task3.dos_victim.pcap --model-dir experiments/  
models/xgb_with_imbalance
```

15 Limitations

- CICIDS2017 has extremely rare classes, especially **Infiltration**.
- Rare-class metrics can fluctuate heavily due to small test support.
- PCAP testing is affected by domain shift.
- Flow extraction may not perfectly match the CICIDS2017 feature generation process.
- KNN has high inference cost on large datasets.
- Logistic Regression does not converge well on the full dataset.

16 Future Work

Future improvements include adding a dedicated CSV-flow prediction command, improving PCAP feature extraction, adding probability thresholding for XGBoost, adding SHAP-based explainability, separating binary detection from multi-class classification, trying LightGBM or CatBoost, evaluating more external PCAP datasets, and adding clearer experiment tracking.

17 Conclusion

This project successfully implements a full Machine Learning-based NIDS pipeline using CICIDS2017. It supports data loading, cleaning, label grouping, imbalance handling, model training, model evaluation, artifact saving, and offline PCAP inference.

The experiments show that Random Forest achieves the best internal CICIDS2017 full-dataset Macro-F1 score. However, XGBoost with conservative imbalance handling provides the strongest evidence that imbalance handling improves rare attack detection, especially for **Infiltration**. In PCAP inference, XGBoost is the most useful model among the tested models because it is more sensitive to attack-like flow patterns under model-only inference.

The project demonstrates an important real-world lesson: high test performance on a benchmark dataset does not automatically guarantee strong performance on external PCAP traffic. Domain shift and flow extraction differences matter. Therefore, the project should present Random Forest as the strongest internal benchmark model and XGBoost with imbalance handling as the most practical model for minority-class and PCAP-oriented analysis.